Step 0a — Come ottenere l'evidenza alla velocità di un decision model

Obiettivo

Scegliere come produrre, insieme al verdetto, le frasi del documento che supportano il claim, con il vincolo di **un solo forward e nessun token generato**. Questo file confronta le alternative, spiega perché vengono scartate e affronta il caso dell'evidenza sparsa.

1. Il budget di tempo

Un decision model costa **un prefill**: legge il prompt e produce i logit di una posizione. Stima per Winnow-12B bf16 su una L40S:

Operazione	Costo indicativo	Perché
Prefill di ~2.000 token	~150–300 ms	$\sim 2 \cdot 12$ miliardi $\cdot$ 2.000 $\approx$ 48 TFLOP, a una frazione dei ~360 TFLOPS bf16 della scheda
Un token generato	$\geq$ ~25 ms	ogni passo rilegge ~22 GB di pesi; banda ~864 GB/s
8 token di citazione	~200 ms	quasi un secondo prefill
Un layer lineare su n stati nascosti	microsecondi	

Ogni soluzione che **genera** o che fa **più forward** fuori da questo budget.

2. Le alternative

Opzione	Forward	Token generati	Evidenza sparsa	Fedeltà	Esito
Ablazione: togliere una frase alla volta	n + 1	0	sì (con ricerca greedy)	alta per costruzione	**scartata in produzione**; usata **offline** per etichettare i dati (02b) e come riferimento (01)
Citazioni generate (`Answer: B [S1][S40]`)	1	3–10	sì	buona se vincolata agli ID	**scartata**: +50–100% di latenza
Gradiente × input sul margine	1 + backward	0	debole	bassa	scartata: backward $\approx 2\times$ forward, mappe rumorose
Attenzione da `Answer:` verso le frasi	1	0	sì	discutibile	**baseline** senza training (01)
Testa lineare per frase	1	0	**no** (§3)	buona	base della soluzione
Pointer $q^\top W h_k$	1	0	parziale	buona	variante più semplice (07, H3)
Testa + aggregatore bidirezionale	1	0	**sì**	buona	**scelta**

3. Il problema dell'evidenza sparsa

In un modello decoder lo stato alla fine della frase k vede solo i token **precedenti**.

Stato	Vede	Conseguenza
`h_1` (prima frase)	istruzioni, claim	non sa se più avanti c'è una frase che la completa
`h_40` (ultima)	claim, S1 … S39	sa che S1 c'era e che insieme stabiliscono il claim
`q` (posizione `Answer:`)	tutto	sa quale catena regge il verdetto

Esempio dove la testa per frase sbaglia:

> Claim: "Nancy R. Howell teaches at a seminary in Kansas City." S1: "Saint Paul School of Theology is a seminary located in Kansas City, Missouri." S40: "Nancy R. Howell is Professor of Theology at Saint Paul School of Theology."

`h_1` vede il claim e S1: "seminary" e "Kansas City" coincidono, quindi qui la testa può ancora indovinare. Ma con una frase ponte che non nomina nulla del claim (per esempio "Saint Paul School of Theology moved to Leawood in 2013", quando il claim parla di Leawood solo attraverso S40), `h_1` non ha nessun indizio.

Soluzione: un aggregatore non causale sopra gli stati delle frasi.

```
x_0 = P_q · q,   x_k = P_h · h_k + E_pos[k]        (proiezioni d → 256, posizione della frase)
X   = TransformerEncoder(2 layer, 4 teste, 256) ( [x_0, x_1, …, x_n] )      # attenzione piena
P(evidenza_k) = σ( v · X_k + b )
```

- Le frasi si vedono a vicenda: S1 "sa" di S40.
- `x_0` porta lo stato finale, che conosce il verdetto e la catena.
- Parametri: ~1,5M (2 layer da 256), contro 12B del backbone.
- Costo: per n = 200 frasi, ~2 · 1,5 milioni · 200 ≈ 0,6 GFLOP: trascurabile.

4. Perché il claim va prima del documento

Se il claim viene dopo il documento (ordine nativo di Winnow e JPT), nessuno stato `h_k` conosce il claim: tutta l'informazione sul claim arriva solo tramite `q`. L'aggregatore può in parte compensare, ma deve fare da solo il confronto frase–claim con vettori che non sono stati calcolati in funzione del claim.

Con il claim prima: - ogni `h_k` è già "una frase letta sapendo cosa si verifica"; - il verdetto si legge sempre alla fine, dove il modello vede tutto: non perde nulla.

Costo: un cambio di formato rispetto al prompt nativo. Va misurato zero-shot (01, B-ORD) e poi è assorbito dal fine-tuning del verdetto, che in questo piano si fa direttamente con il claim prima (02d). L'esperimento H4 (07) misura la differenza con l'ordine nativo.

5. Perché niente chunking

Con il chunking e la regola `max` (MiniCheck, FactCG), se S1 e S40 stanno in chunk diversi: - nessun chunk da solo supporta il claim → verdetto "non supportato"; - e l'evidenza non esiste.

Per questo piano il documento passa **intero** in un forward, entro il contesto del modello (Winnow e JPT gestiscono decine di migliaia di token). Il costo del prefill cresce con la lunghezza, ma resta un solo forward. Documenti oltre il contesto: fuori dal perimetro (vanno spezzati a monte, sapendo che l'evidenza fra pezzi diversi si perde).

6. Granularità: frasi, non token

Unità	Pro	Contro
Frase	annotazioni disponibili (FEVER, HoVer, SciFact); leggibile per l'utente	frasi lunghe evidenziano anche parti irrilevanti
Token / span	preciso	quasi nessuna annotazione a questa granularità per il supporto; più rumoroso

Si parte dalle frasi. Le frasi molto lunghe (> 60 parole) si spezzano su `;` e `:` in fase di segmentazione (03).

7. Raccomandazione

1. Verdetto dal piano binario, addestrato con il **claim prima del documento**.
2. Testa di evidenza = aggregatore bidirezionale a 2 layer su `[q, h_1 … h_n]`, dagli stati di fine frase dello stesso forward.
3. Training in due tempi: testa su backbone **congelato** (il verdetto non cambia); se non basta, LoRA e testa insieme con il verdetto sorvegliato.
4. L'ablazione con n forward si usa solo offline: per etichettare dati e come riferimento di qualità.

Criteri per passare allo step successivo

- Architettura approvata; deciso l'ordine claim → documento.
- Stabilito il budget di latenza (+3%) e la lunghezza massima del documento in produzione.